NeuroDetect

Detailed Implementation Report

A complete technical record of the implemented cognitive assessment prototype

Coverage: implemented frontend flow, FastAPI backend, Supabase schema and persistence, mental-health scoring, five cognitive tasks, 47-feature Model 1 integration, result aggregation, validation, and test verification.

Document status	Detailed implementation record based on the current project codebase.
System type	Browser-based research/demo cognitive assessment prototype.
Core identifier	One assessment_id links the patient, screening, tasks, prediction, and result page.
Report date	September 2026
Clinical note	Outputs are not presented as a diagnosis and must not be used as a substitute for clinical judgement.

Contents

- 1. Project Scope and Implementation Summary
- 2. System Architecture
- 3. End-to-End Assessment Process
- 4. Frontend Implementation
- 5. Backend, APIs, and Validation
- 6. Supabase Data Model and Persistence
- 7. Mental Health Screening
- 8. Cognitive Task Implementation
- 9. Feature Engineering and Formula Reference
- 10. Model 1 Integration and 47-Feature Contract
- 11. Final Results Aggregation
- 12. Reliability, Testing, and Current Limitations
- Appendix A. Feature Bifurcation
- Appendix B. API and Database Inventory

> This report intentionally separates **implemented functionality** from **future scope**. MRI processing, speech analysis, multimodal fusion, diagnostic classification, Grad-CAM, SHAP, and progression prediction are not implemented in the current codebase.

1. Project Scope and Implementation Summary

NeuroDetect is a web-based cognitive assessment prototype designed to capture structured cognitive and behavioural information in one assessment session. The delivered implementation focuses on reliable collection, backend-owned feature extraction, persistence, a cognitive-performance Model 1 integration, and an aggregated final result view.

Implemented capabilities

- Patient registration and assessment-session creation with a generated UUID assessment_id.
- PHQ-9 and GAD-7 questionnaires with total-score calculation, severity bands, and an evaluation cutoff flag.
- Five integrated cognitive tasks: MMSE, Memory Recall, N-Back, Boston Naming, and Trail Making.
- Raw task telemetry submitted by the frontend and transformed into authoritative feature values by FastAPI services.
- Supabase/PostgreSQL storage for patient, assessment, screening, game, event, and Model 1 result data.
- A fixed-order 47-feature vector for Model 1 cognitive-performance regression.
- A final results endpoint and frontend page that remains usable after refresh or direct navigation because results are reloaded from persisted backend data.

Design principles applied

Principle	Implementation decision	Why it matters
Single source of truth	Backend calculates authoritative features and stores results.	Avoids browser-only state being treated as clinical/research data.
Session linkage	All modules carry the same assessment_id.	Preserves a coherent, auditable assessment record.
Layered architecture	UI -> API router -> service -> repository -> Supabase.	Keeps validation, business logic, and persistence separated.
Safe presentation	Results use non-diagnostic language and show unavailable future components as pending.	Prevents overclaiming system capability.

2. System Architecture

The implementation follows a client-server architecture. The React application handles interaction and telemetry capture. FastAPI validates payloads, calculates features, and persists records through repository classes. Supabase provides the PostgreSQL data store.

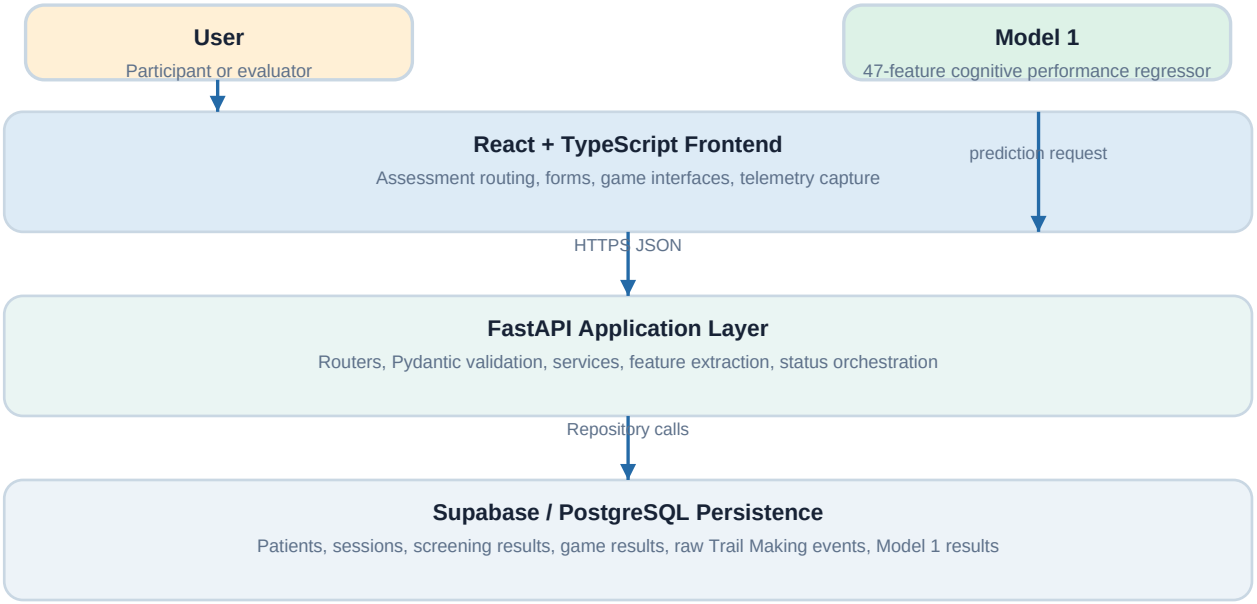

Figure 1. Layered system architecture and the Model 1 integration point.

Layer responsibilities

Layer	Implemented responsibility
React + TypeScript	Multi-step assessment route flow; patient and screening forms; five game interfaces; timing and interaction telemetry; retry/error states; final results display.
FastAPI routers	REST endpoints for assessment creation, screening completion, game completion, Model 1 prediction, and result retrieval.
Service layer	Assessment state control, score calculation, feature extraction, fixed-schema aggregation, error translation, and response shaping.
Repository layer	Isolated Supabase queries and inserts/upserts; keeps persistence details out of API and calculation code.
Supabase/PostgreSQL	Relational tables, UUID keys, referential relationships, uniqueness rules, database-level constraints, and JSON payload fields for rich telemetry.

The frontend does not directly connect to Supabase and does not compute the final authoritative feature values. This keeps the persisted research record consistent with the backend's validation and feature logic.

3. End-to-End Assessment Process

The system uses a session-first workflow. A patient record is created, then an assessment record is created. The resulting assessment_id is passed through every subsequent route and is the foreign key used by all result tables.

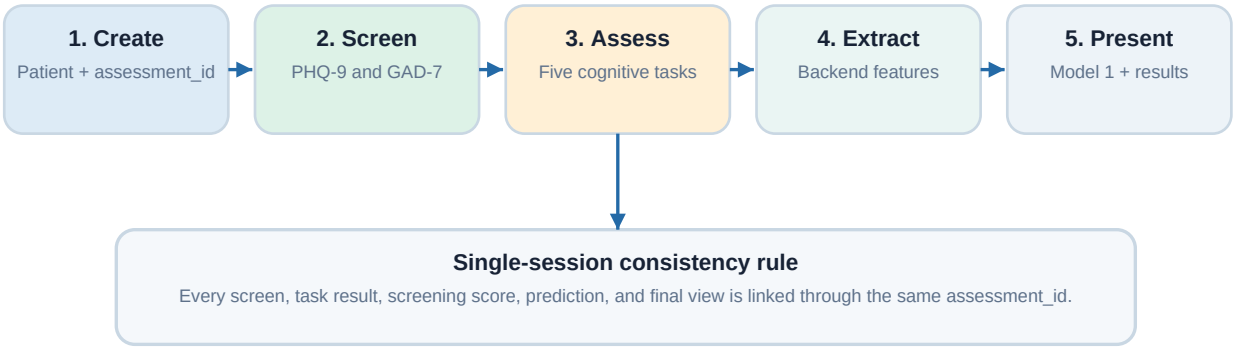

Figure 2. Assessment process and the role of the shared assessment_id.

Implemented sequence

Step	Module	Implementation
1	Create session	Patient details are submitted. The backend creates patients and assessments records and returns assessment_id.
2	Mental health	PHQ-9 and GAD-7 answers are scored and persisted once per assessment.
3	Cognitive tasks	MMSE, Memory Recall, N-Back, Boston Naming, and Trail Making are completed in sequence.
4	Feature extraction	Each game completion payload is validated and transformed into task-specific numerical features.
5	Model 1	After all five task records exist, the aggregator builds and validates the 47-feature vector before prediction and storage.
6	Results	The result service loads all persisted components, checks completion, and returns a stable response for the final page.

Assessment-state handling

The assessment session stores completed stages and a current stage. The Results service derives a completion checklist for PHQ-9, GAD-7, each game, and Model 1. This means the page can identify an incomplete session, an assessment waiting for a Model 1 result, or a completed assessment without relying only on browser navigation state.

4. Frontend Implementation

The frontend is implemented in React and TypeScript. The main application route flow guides a participant from the start screen through patient registration, screening, the five cognitive tasks, and the result page.

Route and user-experience implementation

Area	Implemented work
Assessment routing	Home -> Patient -> Mental Health -> Cognitive Assessment -> MMSE -> Memory Recall -> N-Back -> Boston Naming -> Trail Making -> Results.
Shared context	The returned assessment_id is carried into task routes and API requests to maintain one linked assessment session.
Reusable UI	Application shell, assessment progress display, game shell, buttons, loading states, and failure states reduce repeated layout logic.
Task telemetry	Games capture task-specific events, responses, timestamps, retries, clicks, pointer movement samples, and completion payloads.
Results resilience	The Results screen requests persisted data from the backend. Refreshes and direct result URLs do not depend on in-memory frontend state.
User feedback	Loading, incomplete, retry, and server-error states are exposed rather than silently showing incorrect or stale data.

Frontend API boundary

The API client in frontend/src/services/api.ts owns communication with the backend. It sends typed payloads to the assessment, screening, game, Model 1, and results endpoints. The browser collects interactions; the backend owns validation and derived values.

> A key implementation improvement was changing the result page from navigation-dependent presentation to persisted-data presentation. This directly addresses the common failure mode where a refresh loses a client-only result.

5. Backend, APIs, and Validation

FastAPI exposes the application API. The backend is divided into feature-oriented modules and uses Pydantic request schemas, service classes, and repositories. This separation keeps game logic testable and prevents persistence calls from being mixed with presentation code.

API module design

Module	Role
assessments	Creates the patient and assessment session; controls stage-aware session state.
mental_health	Receives PHQ-9/GAD-7 answers, calculates score/severity, checks valid UUIDs, and avoids duplicate records.
games/mmse	Validates MMSE event payloads and derives orientation, attention, recall, language, timing, omission, and consistency measures.
games/memory	Processes learning, immediate recall, delayed recall, recognition, confidence, retry, and latency data.
games/nback	Computes accuracy, hit/false-alarm metrics, d-prime, response consistency, fatigue trend, and time measures.
games/boston_naming	Processes naming responses, cue use, error categories, typing response durations, and difficulty-aware accuracy.
games/trail_making	Calculates path, sequence, switching, backtracking, entropy, trajectory, and jitter features; also persists sampled raw telemetry.
ml/model1	Fetches all required game records, builds the exact 47-feature vector, validates it, runs Model 1, and stores the result.
results	Aggregates persisted status, screening, Model 1 metadata/result, and component completion into a frontend-safe result response.

Validation and failure behaviour

- Assessment IDs are validated as UUIDs before dependent work is performed.
- Feature aggregation requires all five game results. Missing components produce an explicit incomplete-assessment error rather than a partial Model 1 vector.
- The 47-feature vector is checked for exact ordered schema, numeric values, finite values, and non-zero denominators before model inference.
- Screening results are idempotent: a completed PHQ-9/GAD-7 pair can be returned as already completed instead of inserted again.
- The results response exposes incomplete and awaiting-model statuses, enabling the frontend to show actionable retry guidance.

6. Supabase Data Model and Persistence

Supabase PostgreSQL stores the assessment as a linked relational record. UUID primary keys and foreign keys connect patient, session, screening, game result, telemetry, and Model 1 rows.

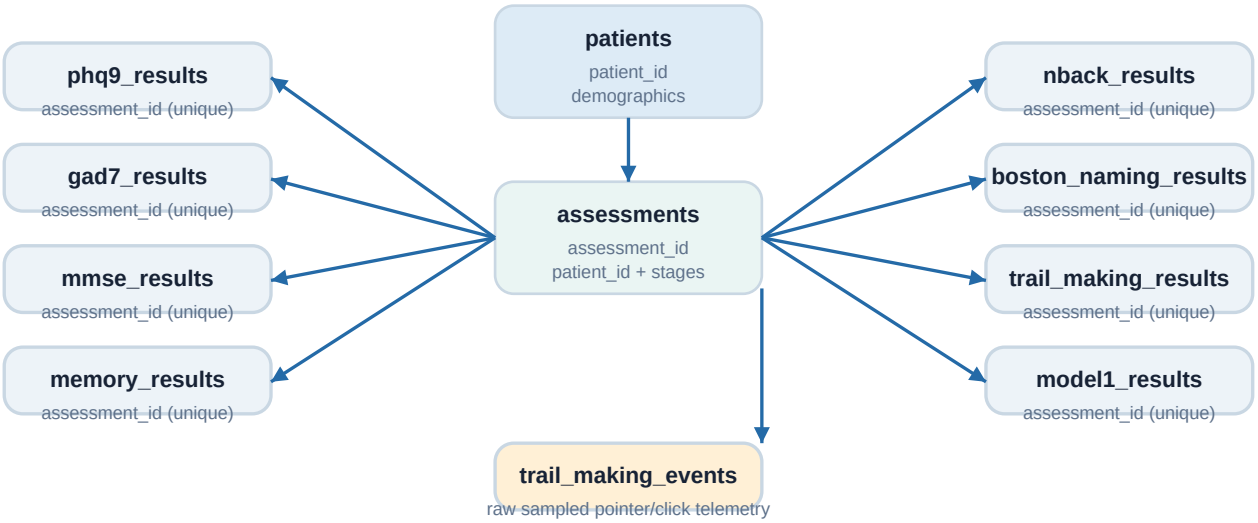

Figure 3. Core persisted entities. Result tables are linked to assessments through assessment_id.

Persistence controls

Control	Implementation
Primary identifiers	UUID primary keys are created with pgcrypto/gen_random_uuid().
Referential integrity	Every result record references assessments(assessment_id) with on delete cascade.
One result per task	Each task result table enforces a unique assessment_id constraint.
Input ranges	SQL CHECK constraints enforce valid score, ratio, count, time, and non-negative value ranges.
Query performance	Indexes are defined for assessment_id and for Trail Making telemetry queries.
Flexible telemetry	Raw nested payloads/layout are retained in JSONB where a fixed relational shape would lose useful interaction detail.

Stored entities

The current schema includes patients, assessments, phq9_results, gad7_results, mmse_results, memory_results, nback_results, boston_naming_results, trail_making_results, trail_making_events, and model1_results. The final Results endpoint is deliberately computed from these persisted records rather than from a separate duplicate results table.

7. Mental Health Screening

The mental-health module implements PHQ-9 and GAD-7 as structured questionnaires. Each response is an integer in the range 0 to 3. The backend calculates total score, severity category, and a boolean indicating whether the score meets the configured evaluation cutoff.

Scoring equations

Instrument	Equation	Used in the system
PHQ-9	PHQ9_total = sum(q1, q2, ..., q9), where qi in {0,1,2,3}	Stored in phq9_results.total_score; maximum = 27.
GAD-7	GAD7_total = sum(q1, q2, ..., q7), where qi in {0,1,2,3}	Stored in gad7_results.total_score; maximum = 21.
Evaluation flag	evaluation_cutoff_met = (total_score >= 10)	Returned for both instruments to signal the configured review threshold.

Severity-band implementation

Scale	Score range	Severity returned
PHQ-9	0-4 / 5-9 / 10-14 / 15-19 / 20-27	Minimal / Mild / Moderate / Moderately severe / Severe
GAD-7	0-4 / 5-9 / 10-14 / 15-21	Minimal / Mild / Moderate / Severe

> The current implementation reports screening scores and severity as assessment context. It does not classify pseudo-dementia, diagnose psychiatric conditions, or determine whether cognitive performance is neurodegenerative or psychiatric-related.

8. Cognitive Task Implementation

The five tasks are implemented as browser-based interactions. Each task produces a completion payload that is validated and transformed into a structured feature record by the backend. The following table describes the implemented scope.

Task	Primary cognitive area	Captured implementation data	Persisted outcome
MMSE	Orientation, registration, attention, recall, language, command following.	Phase answers, correctness, response timestamps, corrections, omissions, completion timing.	Domain accuracies plus timing, error, correction, and consistency features.
Memory Recall	Encoding, immediate recall, delayed recall, recognition, sequence retention.	Target words, recalled words, response gaps, recognition choices, confidence ratings, retries.	Accuracy, decay, latency, errors, hesitation, confidence variability, and consistency trend.
N-Back	Working memory and sustained attention.	Trial targets, responses, hits, false alarms, response latency, first/second-half performance.	Accuracy, signal-detection measures, timing variation, consistency, omission/commission errors, fatigue.
Boston Naming	Lexical retrieval and naming performance.	Image responses, cues, semantic/phonemic errors, response timing/duration, self-corrections.	Naming/cue/error rates, latency features, difficulty-adjusted accuracy, response consistency.
Trail Making	Visual attention, sequencing, set shifting, psychomotor control.	Ordered target clicks, timestamps, sampled pointer coordinates, error clicks, layout and paths.	Path, sequence, switching, backtracking, entropy, deviation, jitter, and raw telemetry.

Backend-owned feature extraction

The frontend is responsible for recording what happened during the task. It does not declare the final feature values. Backend feature functions compute the saved measures from the raw payload so the same logic is used consistently regardless of browser state or UI retries.

9. Feature Engineering and Formula Reference

The formulas below are the implemented feature calculations or the direct transformations used to create Model 1 inputs. For a mean, standard deviation, and ratio, only valid observed values are used and denominator checks are performed where required.

9.1 Common notation

Symbol	Meaning
N	Number of valid trials, responses, opportunities, or time segments for the relevant task.
C	Number of correct responses.
RTi	Latency/reaction time for valid response i.
mean(x)	Arithmetic mean of observed values.
stdev(x)	Population standard deviation used for variability-related features.
max(0, x)	Lower-bound safeguard so duration-like outputs cannot be negative.

9.2 MMSE features

Feature	Formula	Purpose / use
Domain accuracy	domain_accuracy = correct_in_domain / domain_opportunities	Orientation, registration, attention, serial-7, recall, language, and command-following measures.
Response latency	response_latency = mean(submitted_at - started_at)	Average MMSE response speed.
Latency variability	latency_variability = stdev(RT1, ..., RTN)	Stability of response timing.
Correction frequency	correction_frequency = corrected_responses / response_opportunities	Model 1 input and error recovery proxy.
Performance consistency	performance_consistency = 1 - stdev(domain_accuracies)	Cross-domain stability proxy.
Overall MMSE score	10*orientation + 3*registration + 5*attention + 3*recall + 6*language + 3*command	30-point reconstructed score used by Model 1.
MMSE hesitation	response_latency * max(0, 1 - performance_consistency)	Derived Model 1 feature.
MMSE skip frequency	omission_errors / 30	Derived Model 1 feature over response opportunities.
Error recovery time	response_latency * correction_frequency	Derived Model 1 proxy.

9.3 Memory Recall features

Feature	Formula	Purpose / use
Immediate recall accuracy	correct_immediate_targets / total_targets	Encoding and immediate retrieval.
Delayed recall accuracy	correct_delayed_targets / total_targets	Delayed retrieval.
Recognition accuracy	correct_recognition_choices / total_recognition_items	Recognition discrimination.
Recall decay rate	(immediate_recall_accuracy - delayed_recall_accuracy) / immediate_recall_accuracy	Memory loss between immediate and delayed recall.
Recall latency	first_delayed_response_time - delayed_phase_start	Time to initiate delayed recall.
Sequence recall accuracy	order_preserved_adjacent_pairs / recalled_adjacent_pairs	Order retention.
Hesitation duration	sum(gap_i - threshold for every gap_i > threshold)	Pauses beyond the task hesitation threshold.

Feature	Formula	Purpose / use
Confidence variability	stdev(confidence_immediate, confidence_delayed, confidence_recognition)	Stability of reported confidence.
Memory consistency trend	targets_correct_in_both / targets_correct_immediately	Retention consistency across recall phases.

9.4 N-Back features

Feature	Formula	Purpose / use
N-Back accuracy	correct_scored_trials / total_scored_trials	Overall working-memory task accuracy.
Hit rate	hits / target_trials	Correct response to target trials.
False-alarm rate	false_alarms / non_target_trials	Incorrect responses to non-target trials.
Sensitivity	hit_rate - false_alarm_rate	Simple target/non-target separation measure.
Adjusted d-prime	z((H + 0.5)/(T + 1)) - z((FA + 0.5)/(NT + 1))	Signal detection discrimination; correction prevents infinite z values.
Response consistency	1 / (1 + stdev(valid_RTs) / mean(valid_RTs))	Inverse coefficient-of-variation consistency measure.
Fatigue trend	second_half_accuracy - first_half_accuracy	Change in accuracy during the task.
N-Back idle time	max(0, completion_time - response_latency * (hit_rate*T_est + false_alarm_rate*NT_est))	Model 1 inactive-time proxy.
Missed target rate	min(1, omission_errors / T_est)	Model 1 omission measure using estimated target trials.

9.5 Boston Naming features

Feature	Formula	Purpose / use
Naming accuracy	correct_final_responses / total_trials	Overall lexical naming performance.
Cue rates	cue_type_count / total_trials	Semantic, phonemic, and total cue dependency.
Mean response latency	mean(response_latency_ms) / 1000	Model 1 response-latency input in seconds.
Difficulty-adjusted accuracy	sum(weight_i * correct_i) / sum(weight_i), weights: easy=1, medium=1.25, hard=1.5	Difficulty-sensitive task accuracy.
Latency consistency	1 / (1 + stdev(latencies) / mean(latencies))	Response timing stability.
Response signature consistency	unique_response_signatures / total_response_signatures	Diversity/repetition proxy across responses.
Naming consistency	mean(latency_consistency, response_signature_consistency)	Combined consistency measure.
Typing/speaking speed	mean(number_of_words / response_duration_seconds)	Model 1 linguistic response-speed feature.
Time per image	(completion_time_ms / total_trials) / 1000	Model 1 per-item time feature.

9.6 Trail Making features

Feature	Formula	Purpose / use
Path efficiency	ideal_path_length / actual_path_length	How directly the participant traversed the expected path.
Sequence accuracy	correct_target_count / total_targets	Correctness of alternating sequence selections.
Switching delay	mean(correct_target_RTs after first target)	Set-shifting speed after task initiation.
Backtracking frequency	backtracking_count / total_targets	Reversal/correction behaviour.
Cursor movement entropy	-sum(p(direction) * log2(p(direction)))	Directional movement dispersion.
Trajectory deviation	mean((actual_segment - ideal_segment) / ideal_segment)	Deviation from the ideal sequence path.

Feature	Formula	Purpose / use
Tremor/jitter frequency	(direction_reversals + micro_movements) / total_time_seconds	Fine movement instability proxy.
Time degradation ratio	second_half_mean_RT / first_half_mean_RT	Timing change over task progression.

These feature equations support research/demo scoring. They are not clinical diagnostic thresholds, and the prototype does not infer disease labels from any single formula.

10. Model 1 Integration and 47-Feature Contract

Model 1 is integrated as a cognitive-performance regressor. It is only invoked after all five cognitive task result records are available. The feature aggregator reads the saved records, derives any required transformations, and enforces a fixed 47-feature schema before inference.

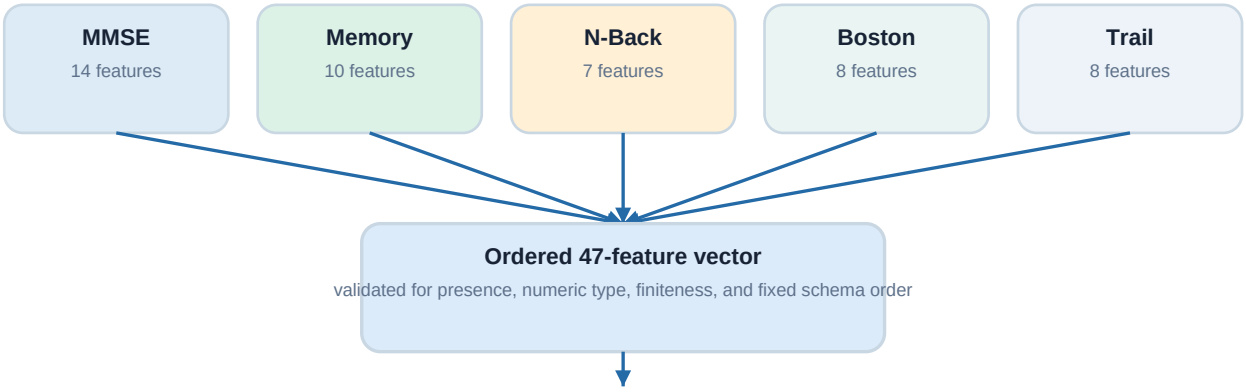

Figure 4. Task features are assembled into one ordered Model 1 vector.

Inference representation

Model 1 prediction: cognitive_performance_score = f_XGBoost(x1, x2, ..., x47). The exact prediction is produced by the trained serialized model, so it does not reduce to a single manually calculated linear equation. The implemented code validates the ordered input vector before calling the model and stores the score with model metadata and the feature vector.

Model safeguards

- The vector must have exactly 47 fields in the expected order.
- Every value must be numeric and finite.
- Required task records must exist before aggregation; otherwise the API returns the missing game names.
- Derived ratios use safe division and reject zero denominators rather than silently emitting invalid values.
- The stored model1_results row keeps assessment_id, cognitive_performance_score, model name/version, and the feature_vector JSON payload.

11. Final Results Aggregation

The final page is backed by a dedicated Results service. Instead of trusting a locally-held frontend result, it loads the persisted assessment, screening results, Model 1 result, and stage completion state using assessment_id.

Returned result components

Component	Displayed when available	Current status
Assessment summary	Participant name, short assessment ID, completed/missing component checklist.	Implemented
PHQ-9	Total score out of 27 and severity label.	Implemented
GAD-7	Total score out of 21 and severity label.	Implemented
Model 1	Cognitive performance score, model name/version, feature count, model type, and scale metadata.	Implemented
Model 3	Speech assessment placeholder and description.	Explicitly pending
Fusion	Overall multimodal result placeholder and description.	Explicitly pending

Completion logic

Results status	Condition	Frontend meaning
completed	All screening, game, and Model 1 components exist.	Show available results and completed assessment summary.
awaiting_model1	All non-Model 1 components exist but no stored Model 1 result exists.	Assessment data is complete; prompt retry for Model 1.
incomplete	One or more required components are absent.	Show missing component information instead of an invalid score.

The language used in the result experience is intentionally non-diagnostic. The system presents assessment scores and a research/demo cognitive performance output, not a diagnosis of Alzheimer’s disease, MCI, pseudo-dementia, or any mental health condition.

12. Reliability, Testing, and Current Limitations

The project includes verification at the frontend build and backend API/service levels. Database constraints and explicit status responses add protection against missing data and invalid model inputs.

Completed verification

Area	Verification performed	Outcome
Frontend	Production build: npm run build.	Passed.
Backend results	Router and service tests for result aggregation and status behaviour.	Passed.
Assessment/Model 1	Assessment router and Model 1 endpoint tests in the targeted test run.	Passed.
Targeted test run	python -m pytest tests/results tests/assessments/test_router.py tests/model1/test_router.py -q.	14 passed, 1 warning at the time of verification.

Current limitations and accurately-scoped future work

- This is a research/demo prototype and not a validated clinical diagnostic instrument.
- No authentication or role-based access control is currently implemented; CORS is permissive for development and must be tightened for deployment.
- Live Supabase validation depends on applying the current SQL schema in the target Supabase project.
- End-to-end browser automation is not yet included.
- Trail Making pointer data is sampled to balance telemetry detail and browser overhead; it is not a capture of every movement event.
- Speech assessment (Model 3), multimodal fusion, MRI analysis, disease-stage classification, explainability heatmaps/SHAP, and longitudinal progression prediction are future work, not current features.

Recommended next implementation steps

Before wider use, add authenticated access, restrict CORS, apply migrations through a controlled environment, add end-to-end tests, introduce audit logging, establish consent/data-retention processes, validate the model on appropriate held-out data, and obtain clinical/research governance review for any real participant deployment.

Appendix A. Model 1 Feature Bifurcation

The 47 input columns are grouped below by task. This appendix mirrors the aggregator contract and is useful for implementation review and PPT preparation.

Order	Task group	Feature name
1	MMSE - 14	overall_mmse_score
2	MMSE - 14	orientation_score
3	MMSE - 14	attention_score
4	MMSE - 14	recall_score
5	MMSE - 14	language_score
6	MMSE - 14	question_accuracy
7	MMSE - 14	mean_reaction_time
8	MMSE - 14	reaction_time_variance
9	MMSE - 14	hesitation_duration
10	MMSE - 14	correction_frequency
11	MMSE - 14	skip_frequency
12	MMSE - 14	confidence_variability
13	MMSE - 14	completion_time
14	MMSE - 14	error_recovery_time
15	Memory Recall - 10	immediate_recall_accuracy
16	Memory Recall - 10	delayed_recall_accuracy
17	Memory Recall - 10	recognition_accuracy
18	Memory Recall - 10	recall_decay_rate
19	Memory Recall - 10	recall_latency
20	Memory Recall - 10	omission_errors
21	Memory Recall - 10	intrusion_errors
22	Memory Recall - 10	sequence_recall_accuracy
23	Memory Recall - 10	retry_frequency
24	Memory Recall - 10	memory_consistency_trend
25	N-Back - 7	nback_accuracy
26	N-Back - 7	false_positive_rate
27	N-Back - 7	missed_target_rate
28	N-Back - 7	sustained_attention_score
29	N-Back - 7	attention_drift
30	N-Back - 7	idle_time
31	N-Back - 7	fatigue_progression
32	Boston Naming - 8	naming_accuracy
33	Boston Naming - 8	response_latency
34	Boston Naming - 8	semantic_error_frequency
35	Boston Naming - 8	phonemic_error_frequency

Order	Task group	Feature name
36	Boston Naming - 8	cue_dependency_rate
37	Boston Naming - 8	no_response_frequency
38	Boston Naming - 8	typing_speaking_speed
39	Boston Naming - 8	time_per_image
40	Trail Making - 8	path_efficiency
41	Trail Making - 8	sequence_accuracy
42	Trail Making - 8	switching_delay
43	Trail Making - 8	backtracking_frequency
44	Trail Making - 8	rule_violation_count
45	Trail Making - 8	cursor_movement_entropy
46	Trail Making - 8	trajectory_deviation
47	Trail Making - 8	tremor_jitter_frequency

Appendix B. API and Database Inventory

Key API endpoints

Method	Endpoint	Purpose
POST	/api/assessments	Create patient and assessment session.
POST	/api/assessments/{assessment_id}/mental-health/complete	Persist PHQ-9 and GAD-7 results.
POST	/api/assessments/{assessment_id}/games/mmse/complete	Persist MMSE-derived result features.
POST	/api/assessments/{assessment_id}/games/memory/complete	Persist Memory Recall-derived result features.
POST	/api/assessments/{assessment_id}/games/nback/complete	Persist N-Back-derived result features.
POST	/api/assessments/{assessment_id}/games/boston-naming/complete	Persist Boston Naming-derived result features.
POST	/api/assessments/{assessment_id}/games/trail-making/complete	Persist Trail Making result features and telemetry.
POST	/api/assessments/{assessment_id}/model1/predict	Build the 47-feature vector, run Model 1, and store the output.
GET	/api/assessments/{assessment_id}/results	Return the persisted assessment summary and component statuses.

Environment and run configuration

Backend variables	SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, ENVIRONMENT
Frontend variable	VITE_API_BASE_URL
Frontend development	frontend: npm install; npm run dev (default localhost:5173)
Backend development	backend: create/activate venv; pip install -r requirements.txt; uvicorn app.main:app --reload --port 8000
Database setup	Apply database/schema.sql, then any required per-game schema files in the Supabase SQL editor.

End of report. The source of truth for the described implementation is the current NeuroDetect repository. This document is designed to support project demonstration, implementation review, and technical presentation preparation.